tmdoc(n) 0.20.1 "Literate programming using Tcl/Tk and various command line tools"

Name

tmdoc - Evaluate embedded R, Python, Julia, Octave and Tcl code or other command line tool input in Markdown, Typst, AsciiDoc or LaTeX documents

Table Of Contents

Table Of Contents
Synopsis
Description
R code Embedding
Tcl code Embedding
Shell Code Embedding
Embedding Programming Languages
Embedding Diagram Code Using Kroki Webservice
LaTeX Equations
Chemical Formulas and Reactions
CSV Tables
Abbreviations
Markdown Alerts
Figure and Table Numbering
Functions
Code Chunk Options
Inline code chunks
Tables
Images
Includes
Command Line Applications
Kroki Webservice
Examples
Filetypes
See Also
Bugs, Ideas, Feedback
Code Copyright
Category
Copyright

Synopsis

package require **Tcl 8.6-**
package require **textutil**
package require **yaml**
package require **fileutil**
package require **cmdline**

::tmdoc::main *argv*
::tmdoc::tmdoc *infile outfile* ?**-mode** *weave|tangle* **-abbrev** *abbrevfile* **-toc** *false|true*?
::tmdoc::tmeval *text* ??*ext md*??

Description

The package **tmdoc** provides a command to evaluate embedded R, Python, Julia, Octave, Tcl and Shell code in code chunks within documents of Markup languages like Markdown, AsciiDoc, Typst or LaTeX and add the resulting output to the document for creating dynamic documents, a technique as well known as literate programming. Further it can use the kroki.io webservice to embed diagrams for this web service if diagram tools like GraphViz or PlantUML are not installed locally. Since version 0.10.0 as well code written in various programming languages can be embedded, compiled and executed making **tmdoc** an useful utility to write programming tutorials. Further since version 0.12.0 LaTeX equations and Youtube videos can be embedded and since version 0.13.0 there is as well support for defining abbreviations in YAML files or in the YAML header, Markdown alerts and embedding tables as CSV code. Conversion of the created Markdown documents can be done by using Tcllib's **Markdown** package.

R code Embedding

The embedded R code starts with three backticks and a language marker like this:

```
```{r}
print("Hello World")
```
```

Shorter R code fragments can be embedded directly within the text using single backticks like this:

```
This document was generated at \
    `r Sys.Date()` MET.
```

Similarly code for the Programming languages Python, Octave and Julia can be embedded. The chunk markers for these languages are py, oc and jl.

Tcl code Embedding

The embedded Tcl code starts with three backticks and a language marker like this:

```
```{.tcl}
puts "Hello World"
```
```

Shorter Tcl code fragments can be embedded directly within the text using single backticks like this:

```
This document was generated at \
    `tcl clock format [clock seconds] -format "%Y-%m-%d %H:%m"` MET.
```

Shell Code Embedding

The embedded Shell code chunks start as well with three backticks then the *.shell* indicator and a *cmd* code chunk argument which explains how to process the input in the code chunk like this:

```
```{.shell cmd="dot -Tpng %i -o%o"}
digraph g {
  A -> B
}
```
```

The arguments *%i* are used as placeholder for the input file name, which contains the text in the code chunk and *%o* for the output filename, usually a PNG or SVG file. Further *%b* can be used as placeholder for the basename of the inputfile, so the input filename without the file extension. If the text from the previous code chunk just needs to be reused again in the next one, you can use the string 'LASTFILE' as a place holder.

Embedding Programming Languages

Programming code can be as well executed, even programming languages requiring an compilation step can be embedded. Here is an example for the Python programming language:

```
```{.shell cmd="python3 %i > %o" chunk.ext=py ext=txt}
print("Hello Python World!")
```
```

And here an example on how to display the output of a C program:

```
```{.shell cmd="gcc %i -o hello&&./hello > %o" chunk.ext=c ext=txt}
#include <stdio.h>
int main() {
    printf("Hello C World!\n");
    return(0);
}
```
```

Please note, that the compilation step is separated by the execution step using the _&&_ and operator of the Bash shell.

Embedding Diagram Code Using Kroki Webservice

Embedded diagram chunks using the kroki.io web service are done like this:

```
```{.kroki dia=graphviz ext=png}
digraph g {
  A -> B
}
```
```

Other file extensions are svg and pdf. Other diagram tools are plantuml, ditaa and many others. For details look at the kroki.io website. Since version 0.14.2 first local availability of the GraphViz dot, the plantuml and the ditaa command line applications are checked before the webservise is used.

LaTeX Equations

Since version 0.12.0 there is as well support for embedding LaTeX equations as SVG images using the math.vercel.app webservice. The equations are hashed as CRC32 hashes and then downloaded if **wget** is installed to avoid refetching the images again and again. if **wget** is not available just the link to math.vercel.app image is embedded. Here an example:

```
```{.mtex echo=false}
E = mc^2
```
```

Chemical Formulas and Reactions

Since version 0.19.0 there is as well limited support for embedding chemical formulas and reactions in the Markdown output. The equations transformed to HTML code using sub and

sup tags, for special features like placing the charge over a number specific css style sheet rules are required.

```
Glucose creation:
`ce 6H2O + 6CO2 -> C6H12O6 + 6O2`
Respiration with backward arrow:
`ce 6CO2 + 6H2O <- C6H12O6 + 6O2`
The dissocation of water showing back and forward arrows is show here:
`ce 2H2O = H3O+ + OH-`
Dissociaton of Natrium Phosphate (requires the css code below, part of tmdoc.css):
`ce Na3PO4 = 3Na+ + PO_4^3-`
Isotop: `ce ^14C`
```

And here the stylesheet code require for 3+ over 4.

```
<style>
/* chem span tag */
.chem {
  font-family: system-ui, sans-serif;
}
/* Normalize size and vertical positions */
.chem sub,
.chem sup {
  font-size: 0.75em;        /* smaller than base text */
  line-height: 0;           /* avoid extra line height */
}
span.pos {
  display: inline-table; /*contain floats*/
  vertical-align: middle;
  margin-left: -0px; /*no whitespace on i element*/
}
/* placement of  _N^N(-|+)  */
.pos sup, .pos sub {
  float: left;
  clear: left;
  font-size: 0.75em;
  line-height: 1;
}
.pos sup { margin-left: -15px; color: red;}
</style>
```

CSV Tables

Since version 0.13.0 there is as well support for embedding CSV data within code chunks and displaying them as Markdown tables. Here an example:

````
```{.csv echo=false results=asis}
col1,col2,col3
1,2,3
4,5,6
7,8,9
```
````

If your values as well contain the separator you most enclose the values within quotes like so:

````
```{.csv echo=false results=asis}
col1,col2,col3
1,"2, 3 or 4",3
4,5,"6, 7 or 8"
7,8,9
```
````

Abbreviations

Since version 0.13.0 there is as well support for embedding abbreviations either in the YAML header of the document or in an external file which must be then referenced in the YAML header like this:

```
---
abbreviations: abbrev.yml
DG: Detlef Groth, University of Potsdam, Germany
---
Inside the of the text the abbreviations can be then expanded using
curly braces like here: This text was written by {DG}!
Here an abbreviation which will be expanded based on the YAML data in abbrev.yml.
{lipsum}
```

For Markup languages which do not support Yaml headers like Typst or AsciiDoc the Yaml file which should be used for abbreviations since version 0.15.0 can be as well given on the command line of the tmdoc application:

```
tmdoc file.ttyp file.typ --abbrev abbrev.yml
tpyst compile file.typ
```

The Tcl procedure **tmdoc::tmdoc** for this purpose has the *-abbrev filename.yml* argument.

Markdown Alerts

Since version 0.13.0 there is as well support for adding Markdown alerts. These alerts are only visible as such by styline the style sheet acordingly. Please have a look at the file **tmdoc.css** for the styling of the alerts which can be seen in the tutorial. Here an example:

```
> [!NOTE]
> This is a note.
> [!WARNING]
> This is a warning!!
```

Figure and Table Numbering

Since version 0.17.1 automatic figure and table numbering is supported using single backticks quoted chunks together with an appropriate label. If the same label appears later in the text the label is reused for the numbering so you have forward and backward referencing for figures and tables. Here an example:

```
The figure `nfig rect` below shows some simple rectangles.
```{.kroki echo=FALSE}
     +----------------+       /---------------- |               |       | cEFF          |
     +----------------+       \----------------/
```
Figure `nfig rect`: Sample rectangle in Ditaa syntax.
```

Functions

::tmdoc::main *argv*

> The main entry point for applications. The *argv* vector is usually the argument vector given on the command line. It is used by the tmdoc command line application but could be used as well by other programming languages liem Python or R which can with an installed Tcl interpreter.

::tmdoc::tmdoc *infile outfile* ?**-mode** *weave|tangle* **-abbrev** *abbrevfile* **-toc** *false|true*?

The command reads the *input* file, evaluates embedded Tcl code in code chunks and writes the resulting output to the *output* file if such is given or to stdout if *output* is missing, this is the default mode *weave*.

If as **-mode** *tangle* is given, the Tcl code in code chunks is extracted but not evaluated.

If as **-abbrev** *FILENAME* is given, the given *abbrevfile* argument should be a YAML file containing abbreviations which can be used within the given *infile*.

If as **-toc** *false|true* is given and set to *true*, there will be a file *file rootname .toc* generated which contains a list with links to all section 2 and 3 headers which can be embedded using the **include** function of tmdoc.

All arguments are paths to the files to read from or write to.

The result of the command is the empty string.

See section Code Chunk Options for the supported code chunk options.

```
package require tmdoc
namespace import tmdoc::tmdoc
tmdoc tmdoc-example.tmd tmdoc-example.md
tmdoc tmdoc-example.tmd tmdoc-example.tcl -mode tangle
tmdoc example.ttyp example.typ --abbrev abbrev..yml
typst compile example.typ
```

::tmdoc::tmeval *text* ?*?ext md?*?

The command evaluates the given *text* as code chunk and returns the resulting output. Note, that the interpreter is reset after each call, so variable declared in previous calls will not exists in calls thereafter.

The given file extension gives the operation mode, so the default *md* assumes that Markdown output is requested, in case of *tdoc* AsciiDoc compatible markup and in case of *ttyp* Typst compatible markup will be returned,.

```
tmdoc::tmeval "
Short code chunk follows:
```{.tcl}
set x 2
set x
```
"
```

This function is mainly used for testing the package and usually not useful in other situations.

Code Chunk Options

The package supports the following code chunk options.

echo

Should the Tcl code itself been show. Defaults to **true**.

eval

Should the code chunk be evaluated. Defaults to **true**.

fig

Should a custom figure function called with the chunk label as argument. Defaults to **false**.

fig.width

The figure width of the image, if not set, the figure width is choosen automatically.

include

Should a possible figure been included directly after the code chunk. Defaults to **true**.

label

A text label for the code chunk. Defaults to **chunk-nn** where nn is a counter.

results

Should the results of code evaluation been show, either **show**, **hide** or **asis**, the latter does include the ouptput not into a pre / verbatim environment. Defaults to **show**.

The following example will just do an computation using the Tcl interpreter but does not show the output due to **results**=**hide** and not the code due to **echo**=**false**:

```
```{.tcl echo=false results=hide}
set date [clock format [clock seconds]]
```
```

Inline code chunks

Short Tcl codes can be as well evaluated within the standard text flow using single backticks. For instance:

```
The current date is  \
  `tcl clock format [clock seconds] -format "%Y-%m-%d"`
```

The code above would insert the current date into the text. So text inline code chunks are possible using single backticks such as here **`tcl set x`**.

Tables

The Tcl interpreter which is executed inside the code chunks supports as well a procedure **list2mdtab headers data** which allows of inserting tables if the output is Markdown.

```
```{.tcl results="asis"}
set l [list [list A B C D E] \
   [list 1 2 3 4 5] \
   [list 6 7 8 9 10] \
   [list 11 12 13 14 15]]
# syntax: list2mdtab header rows
# we need to flatten the row list
set r <center>\n\n
append r [list2mdtab [lindex $l 0] [concat {*}[lrange $l 1 end]]]
append r \n\n</center>\n
```
```

Images

Inside standard code chunks as well images with Tcl can be generated with normal Tcl code.

To support the standard chunk properties **fig**=**true**, optionally with **include**=**false** however a Tcl procedure **figure** must be provided. Below is an example to use the tklib package **canvas::snap** to create images using the Tk canvas.

```
```{.tcl}
package require Tk
package require canvas::snap
```
```

```
pack [canvas .c -background beige] \
   -padx 5 -pady 5 -side top -fill both -expand yes
proc figure {outfile} {
    set img [canvas::snap .c]
    $img write $outfile -format png
}
```

After preparing the canvas and the figure procedure the canvas can be used for making images like in the example below:

````
```{.tcl fig=true results=hide}
.c create rectangle 60 60 90 90  -fill blue
```
````

The code above will create a canvas figure and embeds it automatically.

If you need more control on the figure placement you can use the option **include=false**.

````
```{.tcl label=mfig fig=true results=hide include=false}
.c create rectangle 65 65 85 85  -fill blue
```
````

You can now manually place the figure. The filename of the figure will be automatically created, it is the basename of the tmd-file and the label. So in principle: `basename-label.png`. You can embed the figure using Markdown figure markup such as:

```
![](basename-label.png)
```

Includes

tmdoc supports as well the inclusion of other files into the output document, the Tcl interpreter running into your source code chunks has an **include FILENAME** feature which allows you to insert document code directly within your document. This is especially import for markup languages like Markdown which do not have an include directive per default. Here an example to include an Markdown file into the current document.

```
`include filename.md`
```

If you would like to place the file content within a specific environment, for instance to show some source code with in a pre environment, you can as well give two arguments like this

```
`include filename.md {{<pre class="python">} {</pre>}}`
```

Within code chunks you can as well include source code for the appropriate chunk type using the **#INCLUDE "FILENAME"** syntax. Here an example to include a C source code file and then execute and run that file.

````
```{.shell cmd="gcc %i -o hello2&&./hello2 > %o" chunk.ext=c ext=txt}
#INCLUDE "assets/hello2.c"
```
````

Command Line Applications

Since Version 0.8.0 **tmdoc** as well supports the inclusion of code for tools that can create graphics from the given input such like GraphViz **dot** or **plantuml**. These code chunks have a `.cmd` chunk indication and need to provide as well a cmd chunk option which indicates the required command line to process the input. The following code chunk options are available.

cmd

The command line to process the given code chunk input. There are placeholders for the input filename "%i" and the output filename "%o" available.

echo

Should the code input be shown. Defaults to **true**.

eval

Should the code chunk be evaluated. Defaults to **true**.

fig

Should a custom figure function called with the chunk label as argument. Defaults to **true**.

fig.width

Suggested figure width. Defaults to 0 (automatic mode).

include

Should a possible figure been included directly after the code chunk. Defaults to **true**.

label

A text label for the code chunk. Defaults to **chunk-nn** where nn is a counter.

results

Should the results of code evaluation been show, either **show**, **hide** or **asis**, the latter does include the ouptput not into a pre / verbatim environment. Defaults to **show**.

````
```{.cmd cmd="dot -Tpng -o%o %i" echo=false results=hide}
digraph g {
    A -> B
}
```
````

Using the LASTFILE content of a code chunk the last text of a previous code chunk can be reused. This LASTFILE string can reused several times, here an example which should produce the same image as before just with filled boxes as the **cmd** code chunk option was changed.

````
```{.cmd cmd="dot -Tpng -Nstyle=filled -o%o %i" echo=false results=hide}
LASTFILE
```
````

Kroki Webservice

Since Version 0.9.0 **tmdoc** as well supports the inclusion of code for diagram tools provided by the kroki.io websevices so that you do not need tools like GraphViz **dot** or **plantuml** to be installed locally. Instead in case of an existing internet connection you can fetch images for these tools and many others from the webservice. The following code chunk options are available.

dia

The diagram type, the most common options are graphviz, plantuml, diataa or pkchr, default: ditaa

echo

Should the code input be shown. Defaults to **true**.

eval

Should the code chunk be evaluated. Defaults to **true**.

fig

Should a custom figure function called with the chunk label as argument. Defaults to **true**.

fig.path

the path where the images should be stored, the images are usually downloaded from the webservice locally, default: . (cuurent directory)

include

Should a possible figure be included directly after the code chunk. Defaults to **true**.

label

A text label for the code chunk. Defaults to **chunk-nn** where nn is a counter.

results

Should the results of code evaluation been show, either **show**, **hide** or **asis**, the latter does include the ouptput not into a pre / verbatim environment. Defaults to **show**.

````
```{.kroki dia=graphviz ext=png echo=false results=hide}
digraph g {
    A -> B
}
```
````

Examples

Here is such an example which will be executed by the Tcl interpreter

````
The code chunk below will be executed:
```{.tcl}
set x 1
expr {$x*2}
```
Some more Markup text ...
````

Filetypes

Currently **tmdoc** supports the filetypes Markdown which is assumed if you are using the file extension "**.tmd**", AsciiDoc if you use the "**.tdoc**" extension and LaTeX if you use the file extensions "**.tnw**".

See Also

For a detailed tutorial on how to do literate programming with Tcl have a look at the tmdoc-tutorial.

- tmdoc-tutorial
- tmdoc-application

Bugs, Ideas, Feedback

This document, and the package it describes, will undoubtedly contain bugs and other problems. Please report such to the author of this package. Please also report any ideas for enhancements you may have for either package and/or documentation.

Code Copyright

BSD License type:

The following terms apply to all files a ssociated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

In no event shall the authors or distributors be liable to any party for direct, indirect, special, incidental, or consequential damages arising out of the use of this software, its documentation, or any derivatives thereof, even if the authors have been advised of the possibility of such damage.

The authors and distributors specifically disclaim any warranties, including, but not limited to, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. This software is provided on an "as is" basis, and the authors and distributors have no obligation to provide maintenance, support, updates, enhancements, or modifications.

RESTRICTED RIGHTS: Use, duplication or disclosure by the government is subject to the restrictions as set forth in subparagraph (c) (1) (ii) of the Rights in Technical Data and Computer Software Clause as DFARS 252.227-7013 and FAR 52.227-19.

Category

Text processing

Copyright

Copyright © 2020-2026, Detlef Groth <dgroth(at)uni(minus)potsdam(dot)de>